Table II: SysJava vs SWAT4J on the primary JDK 8 container images (Debian 10, Temurin 8u504).

	Zookeeper 3.4.14 JDK 8u504 Debian 10	Tomcat 7.0 JDK 8u504 Debian 10	Cassandra 3.0.29 JDK 8u504 Debian 10	Jetty 9.4.51 JDK 8u504 Debian 10	Storm 2.4.0 JDK 8u504 Debian 10	OrientDB 3.2.19 JDK 8u504 Debian 10	Elasticsearch 7.5.2 JDK 8u504 Debian 10	Solr 8.7.0 JDK 8u504 Debian 10	Groovy 4.0.0 JDK 8u504 Debian 10	JRuby 9.4.2.0 JDK 8u504 Debian 10
Native Method Resolution										
JNI Total Natives	495	816	975	353	1884	700	626	980	588	598
SysJava Resolved	484	792	903	352	1835	684	609	967	541	587
JNI Static (Java_*)	317	564	596	185	1347	498	427	645	360	416
Partial (_1/$/__)	4	59	31	2	65	18	14	8	7	6
JNI Dynamic (RegisterNatives)	119	122	158	119	366	123	121	261	122	120
JVM	44	47	45	46	47	45	47	47	52	45
JFR/Netty	0	0	73	0	10	0	0	6	0	0
Platform / Intrinsic	10	21	71	0	46	8	16	10	39	3
Not Found	1	3	1	1	3	8	1	3	8	8
SWAT4J Resolved	198	500	534	162	1257	377	287	277	271	366
SWAT4J JNI Static (Java_*)	161	459	388	122	1090	338	247	237	228	327
JNI Partial/Heuristic	0	0	107	0	127	0	0	0	0	0
JVM Source Code	37	41	39	40	40	39	40	40	43	39
Not Found	290	308	434	184	619	315	331	693	308	225
Syscall Allowlist (SysJava only)										
Java Lib Syscalls	108	124	124	92	125	111	113	127	109	95
Container Binary Syscalls	111	92	140	63	101	83	114	111	105	84
Go Whitelist (Δ)	36	35	24	51	26	40	31	28	37	43
Unmapped Lib Syscalls (Δ)	18	0	41	10	2	10	11	20	19	10
Total Syscalls	190	170	230	157	172	173	183	199	189	167
SWAT4J Syscall Comparison										
SWAT4J Java Lib Syscalls	104	105	116	104	106	105	105	103	108	104
SWAT4J Dynamic Syscalls	128	127	141	128	137	129	137	136	130	128
SWAT4J Total (Static ∪ Dynamic)	171	170	179	171	177	169	174	173	175	170
SysJava ∩ SWAT4J	136	134	159	125	138	133	139	144	140	133
SysJava only	54	36	71	32	34	40	44	55	49	34
SWAT4J only	35	36	20	46	39	36	35	29	35	37
Dynamic Capture Artefacts										
Libraries Loaded	20	24	119	15	29	23	35	28	19	19
Binaries Loaded	11	6	23	2	10	6	9	14	12	7
JARs Loaded	10	41	51	16	88	76	182	144	69	9
Directly Mapped Libraries Analysed	11	15	19	8	15	18	15	18	15	7

Note: SysJava Resolved + Platform / Intrinsic + Not Found = JNI Total Natives; SWAT4J Resolved + Not Found + its intrinsic natives = JNI Total Natives (SWAT4J's mapping run on the same native methods and image libraries). Java Lib ∪ Container Binary + Go Whitelist (Δ, runc entries not already counted) + Unmapped Lib (Δ, syscalls of loaded libraries without a Java binding, not already counted) = Total Syscalls. Native methods are found on an RTA call graph (virtual and interface calls dispatched to the overrides of every class instantiated in reachable code). Natives in libraries the image does not ship are mapped to the released library (Solr: Hadoop 3.2.0 libhadoop, Hadoop 3.3.6 libhadoop for the 14 erasure-coding natives, netty epoll 4.1.50, Lucene NativePosixUtil; OrientDB: jffi 1.3.11, NativePosixUtil; Elasticsearch: NativePosixUtil; Groovy: the jansi library bundled in jline 2.14.6; Tomcat: jsvc built from commons-daemon 1.2.4); each library is analysed once from all its mapped start functions. The remaining Not Found natives have no implementation in any library (e.g. jffi Foreign natives never implemented upstream, JFR setCompressedIntegers absent from the 8u504 libjvm). Container binaries are analysed from _start and main; runc is covered by the Go whitelist only. Storm: sync in its generated profile comes only from FFM hits the detector inferred from Clojure class names

(JDK 8 has no FFM API) and is not counted; Storm runs under the 172-syscall profile. Elasticsearch: SysPart cannot analyse the stripped ML controller; its syscalls are recovered from the functions it imports. SWAT4J Java Lib = its static allowlist (JTracer); SWAT4J Dynamic = strace of the container start-up. All ten containers run under their generated profiles.